

4. Locust 활용 부하 테스트

1. Locust: 파이썬 기반의 강력한 성능 테스트 도구

Locust는 파이썬으로 작성되어 복잡한 시나리오도 쉽게 구현할 수 있는 강력한 성능 테스트 도구이다.

1.1. Locust 소개 및 특징

- Locust는 순수 파이썬으로 작성된 성능 테스트 도구이며, 2011년에 공개되어 현재는 커뮤니티 중심으로 관리되고 있다.
- 오픈 소스이며 MIT 라이선스를 따르며, 유료 버전이나 기업 후원은 없다.
- 매달 약 130만 건의 다운로드가 발생할 정도로 큰 규모의 커뮤니티를 자랑하며, Slack 채널, GitHub, Stack Overflow 등에서 활발하게 지원이 이루어진다.
- 발표자는 약 15년간 성능 테스트 분야에서 일했으며, 프리랜서로 '레드 셔츠 컨설팅'을 운영하고 있다.
- Locust를 사용하면 실제 파이썬 코드로 테스트를 정의할 수 있어 복잡한 동작, 반복문, 조건부 동작, 테스트 데이터 생성 등을 쉽게 구현할 수 있다.
- 테스트가 소스 코드로 저장되어 Git을 이용한 협업이 용이하며, 파이썬 기반이라 코드 재사용이 가능하다.
- 훌륭한 마스터-워커 아키텍처를 제공하여 부하 생성기를 분산 실행하고 데이터를 집계한다.
- HTTP를 중심으로 개발되었지만, 추가 애드온이나 직접 작성한 코드를 통해 거의 모든 프로토콜을 테스트할 수 있다.
- Sasei에서 개발한 Locust 플러그인은 웹소켓, Kafka, MQTT, Playwright 등 다양한 추가 사용자 클래스를 제공한다.

1.2. Locust 기본 사용법 및 데모

- Locust 스크립트는 사용자 클래스를 정의하며, 이는 사용자 유형을 나타낸다.
- 사용자 클래스는 사용하는 프로토콜에 따라 기본 사용자 클래스를 상속받으며, Locust가 호출할 작업을 정의한다.
- `client.get` 은 HTTP 사용자로부터 가져온 파이썬 `requests` 라이브러리의 세션 객체 인스턴스로, 익숙한 구문을 사용한다.

- 디버거에서 실행하면 사용자 한 명의 모든 요청을 프롬프트에 출력하며, 중단점 설정도 가능하다.
- 실제 테스트 실행 시에는 웹 UI를 통해 사용자 수, 호스트 등을 설정하고 실행한다.
- 웹 UI는 초당 요청 수, 실패율, 응답 시간, 사용자 수 등을 보여주는 차트를 제공한다.
- 사용자를 점진적으로 늘리면서 응답 시간이 늘어나고 초당 요청 수가 정체되는 현상은 시스템 병목 현상을 의미한다.
- 최신 UI는 Locust 자체 로그를 확인할 수 있는 기능을 제공하여 유용하다.
- 헤드리스 테스트는 명령줄에서 모든 작업을 수행하며, 사용자 수와 실행 시간을 지정하여 진행한다.

1.3. Locust 고급 기능 및 확장

- Locust는 고급 시나리오 구현에 강점을 보이며, MongoDB에서 테스트 데이터를 가져오거나 사용자 지정 명령줄 옵션을 사용할 수 있다.
- 이벤트 핸들러에 기능을 추가하여 요청 실패 시 작업을 다시 시작하도록 설정할 수 있으며, 이는 긴 기능 흐름에서 유용하다.
- Fast HTTP 사용자 인터페이스와 컴파일된 G 이벤트 HTTP 클라이언트를 사용하여 성능을 향상시킬 수 있다.
- '상수 페이싱'은 시스템 응답 시간에 관계없이 일정한 간격으로 작업을 반복하여 일정한 부하를 유지하는 데 중요하다.
- `onstart` 메서드를 사용하여 사용자가 시작할 때마다 특정 작업을 수행하도록 할 수 있다.
- 응답을 조작하거나 특정 조건에 따라 요청을 실패 처리할 수 있으며, 쿠키는 자동으로 처리된다.
- 작업 대기 기능을 사용하여 특정 작업의 실행 확률을 조절할 수 있다.
- 페라리 구매 시나리오처럼 제어 흐름을 만들거나, OpenTelemetry, Jagger 등과 추적 정보를 전송할 수 있다.
- 소켓 IO 사용자 기능은 웹소켓을 사용하며 푸시 알림도 보낼 수 있다.
- 사용자 대기 기능을 통해 특정 사용자 유형의 인스턴스 수를 조절할 수 있다.

1.4. Locust의 한계 및 고려사항

- 파이썬은 컴파일 언어만큼 빠르지 않아 밀리초 단위의 정확도가 요구되는 경우 적합하지 않을 수 있다.

- 전역 인터프리터 잠금(GIL) 때문에 CPU 코어당 하나의 워커 프로세스가 필요하다.
- 코드를 작성하는 것보다 설정을 구성하는 것을 선호하는 사용자에게는 어려울 수 있다.
- 린터와 정적 타입 힌트를 사용하는 것이 좋으며, Locust는 거의 모든 API에 대한 타입 힌트를 제공한다.
- 간단한 테스트 작성은 쉽지만, 고급 시나리오에서 진가를 발휘하며 대규모 사용자 규모로 확장 가능하다.

2. k6와 Grafana: 성능 테스트와 시각화의 결합

k6는 강력한 성능 테스트 도구이며, Grafana는 이를 시각화하여 애플리케이션의 성능을 깊이 이해하도록 돕는다.

2.1. k6 소개 및 기본 사용법

- k6는 Grafana Labs에 합류한 개발자 경험 중심의 성능 테스트 도구이다.
- 성능 테스트는 지연 시간, 트래픽, 오류를 측정하지만, 시스템 포화도나 가시성을 제공하지는 못한다.
- Grafana는 관찰 가능성(Observability)을 제공하고, k6는 데이터를 생성하여 상호 보완적이다.
- k6 스크립트는 JavaScript로 작성되며, `get` 요청과 같은 기본적인 테스트부터 시작할 수 있다.
- 실제 사용자처럼 데이터베이스를 쿼리하는 시나리오를 시뮬레이션할 수 있다.
- 응답 코드(200)와 응답 본문(텍스트 "Pikachu")을 확인하는 `checks` 기능을 사용할 수 있다.
- `think time` 을 추가하여 요청 간격을 조절하고 실제 사용자처럼 보이게 할 수 있다.
- 5분 동안 5명의 사용자로 테스트를 실행할 수 있으며, `k6 run simple.js` 명령어로 실행한다.
- 로컬 실행 시 HTTP 요청 지속 시간(응답 시간)을 확인할 수 있으며, 통과한 요청의 응답 시간만 볼 수도 있다.

2.2. k6 Cloud 및 Grafana 연동

- k6 OSS는 무료이며 단일 머신에서 최대 40,000명의 사용자를 시뮬레이션할 수 있다.
- k6 Cloud는 수만 명의 사용자를 여러 지리적 영역에서 테스트하거나 대시보드를 활용할 때 유용하다.

- `k6 login cloud` 명령어로 로그인 후, `k6 run --out cloud` 또는 `k6 cloud` 명령어로 클라우드에서 테스트를 실행할 수 있다.
- k6 Cloud는 로컬에서 실행된 테스트 결과를 클라우드에서 실시간으로 확인할 수 있는 링크를 제공한다.
- k6 Cloud에서 실행하면 AWS 서버에서 테스트가 진행되며, 스크립트와 CSV 파일이 전송된다.
- k6와 Grafana를 연동하는 두 가지 방법이 있다: k6 Cloud 데이터 소스 플러그인과 Prometheus Remote Write.
- **k6 Cloud 데이터 소스 플러그인:**
 - k6 Cloud를 Grafana의 데이터 소스로 설정하며, k6 OSS에는 적용되지 않는다.
 - Grafana 마켓플레이스에서 플러그인을 설치하고, k6 Cloud 계정의 API 토큰을 입력하여 설정한다.
 - 미리 준비된 대시보드를 가져와 테스트 결과를 빠르게 확인할 수 있다.
 - 테스트 결과 목록, 응답 시간, 체크 통과 여부 등을 시각화한다.
- **Prometheus Remote Write:**
 - k6 OSS 및 k6 Cloud 모두와 호환되며, Prometheus를 활용한다.
 - k6 Cloud UI에서 Grafana Cloud APM 설정을 통해 원격 쓰기 URL, 사용자 이름, API 키를 입력하여 설정한다.
 - 테스트별로 Prometheus Remote Write를 활성화/비활성화할 수 있다.
 - k6 테스트 빌더 UI에서도 Cloud APM 설정을 통해 Grafana Cloud 연동을 활성화할 수 있다.
 - Prometheus Remote Write는 별도의 대시보드를 제공하지 않지만, Grafana에서 Prometheus 데이터 소스를 통해 k6 메트릭을 확인할 수 있다.
- **어떤 방법을 선택해야 할까?**
 - k6 Cloud 데이터 소스 플러그인: k6 Cloud 대시보드를 Grafana에서 그대로 재현하고 싶거나, 과거 테스트 결과를 가져오고 싶을 때 유용하다.
 - Prometheus Remote Write: k6 OSS를 사용하거나, 이미 Prometheus를 사용하는 스택에 k6를 통합하고 싶을 때 적합하다.
- k6와 Grafana를 함께 사용하면 애플리케이션의 내부 동작에 대한 깊은 이해를 얻을 수 있다.